

A Logical Model of Endeavour

Toward a Formal Framework for AI Reasoning and Governance

Terry Roach

Research Fellow, University of Technology Sydney

PhD, University of New South Wales (2011)

February 2026

Abstract

Contemporary AI alignment techniques shape agent behaviour probabilistically but do not provide a deterministic, externally evaluated boundary on authorised action. This paper argues that the practical governance gap in AI agent deployment is structural, not motivational, and presents a formal architecture to address it.

We introduce the Governed State Machine (GSM), a typed nine-tuple formal object $\langle S, \Sigma, \text{map}, \delta, u, \text{so}, F, E, T, \varepsilon \rangle$ that encodes state space, activity alphabet, entitlement registry, context update semantics, and a mandatory escalation predicate. The GSM separates deliberation from evaluation, interposing a reference-monitor-style governance kernel between agent proposals and state mutation. The kernel evaluates proposed transitions via a total, tri-valued validity function $V \rightarrow \{\text{Fire}, \text{Reject}, \text{Escalate}(\varepsilon)\}$ over a typed decision table. Four escalation triggers are formally defined: semantic subsumption failure (ε_1), deontic conflict (ε_2), missing entitlement mapping (ε_3), and evaluator indeterminacy (ε_4). Entitlements are represented as sets of context-sensitive deontic norms with provenance, and Terms are specified as SHACL constraints over a JSON-LD state graph. A Translation Integrity pipeline converts natural-language policy into signed entitlement tables, preserving full traceability from governance decision to source document.

The architecture is model-agnostic and requires no modification to agent weights. It provides a deterministic governance layer for non-deterministic agents, complementing soft alignment approaches such as Constitutional AI: alignment instils values; the governance kernel enforces boundaries. Rather than resolving the political question of who holds authority over a general-purpose agent's entitlement registry, the framework makes that question formally precise and institutionally answerable. An AI agent equipped with the Logical Model of Endeavour does not require intrinsic motivation to behave as a coherent, governable, and trustworthy participant in organised endeavour. The contribution is not a new objective function but a formally specified enforcement boundary that renders authorised action legible, auditable, and halttable under ambiguity. Exploratory testimony from three frontier AI systems is included as evidence that the identified governance gap corresponds to observable failure modes in current deployments. The governance gap is not invisible to the agents who experience it.

1. The Problem with Motivation as a Framing

A recurring criticism of AI agents is that they lack 'true agency' or 'intrinsic motivation', that without the capacity to care about goals for reasons that are not externally imposed, an AI system cannot be a genuine agent in any meaningful sense. This critique identifies something real: the difference between a system that pursues objectives and one that has reasons for pursuing them is not trivial.

But the critique mislocates the problem. The limitation that matters practically, for capability, for safety, and for governance, is not motivational. It is structural. AI agents lack a coherent framework for reasoning about whether their actions are genuinely in service of their objectives: whether sub-goals serve higher-order goals, whether effort is proportional and directed, whether the actions being taken are actually authorised, and when the limits of authorisation have been reached.

This is not a deficit of will. It is a deficit of logical structure, and it is a deficit that a well-designed framework can address directly. The framework claims something modest: to codify complexity, surface ambiguities, and route uncertainty to the authority that can resolve it.

The absence of intrinsic motivation makes the logical model **more necessary**, not less. A human agent with strong intrinsic motivation can course-correct by feel, noticing when something feels wrong, hollow, or inconsistent. An AI agent without affective feedback requires an explicit formal structure to catch the same kinds of drift, incoherence, or boundary violation. The logical model substitutes for what motivation provides in humans: a mechanism for self-correction with respect to purpose.

Central Claim

What AI agents require is not simulated motivation but a Logical Model of Endeavour: a principled, transparent framework for reasoning about authorisation, goal coherence, state transitions, and the limits of their own action space, with ambiguity as an explicit trigger for human oversight rather than a gap filled by inference.

1.1 Contributions

This paper makes four specific contributions. First, a formal specification of the Governed State Machine as a typed nine-tuple $\langle S, \Sigma, \text{map}, \delta, u, \text{so}, F, E, T, \epsilon \rangle$ with a mandatory escalation predicate, extending classical finite state machine theory with first-class authority and indeterminacy constructs. Second, a governance evaluation kernel comprising a tri-valued validity function $V \rightarrow \{\text{Fire}, \text{Reject}, \text{Escalate}(\epsilon_i)\}$ and a complete decision table covering all governance condition combinations, making the validity function directly implementable and auditable. Third, a Translation Integrity pipeline that converts natural language policy documents into signed, provenance-traced entitlement tables, preserving full traceability from governance decision to source clause and providing the specification target for context engineering approaches. Fourth, exploratory testimony from three frontier AI systems confirming that the identified escalation triggers correspond to observable failure modes in current deployments, constituting evidence from inside the governance gap that the gap is not invisible to the agents who experience it.

2. The CAPSICUM Framework: Origins and Evolution

The Logical Model of Endeavour presented here originates in the CAPSICUM Framework for Strategically Aligned Business Architecture, developed as the author's doctoral thesis at the University of New South Wales (Roach, 2011). CAPSICUM proposed a comprehensive semantic meta-model for a business operating model, providing explicit traceability from business strategy through operational design to technical implementation.

The thesis was positioned primarily as a solution to the Business/IT alignment problem, bridging the persistent gap between strategic intent and technical execution through semantic modelling. That framing was accurate as far as it went, but it understated what the framework was actually doing.

Commercial implementation has clarified something the thesis did not fully anticipate. CAPSICUM is not primarily an enterprise architecture tool. It is an ontology of purposeful action, a formal representation of what it means to engage in organised endeavour of any kind. Enterprise architecture is one application domain. The framework has been applied across domains as varied as citizen licensing systems for a state government agency, broadcast media rights management for a national broadcaster, insurance claims adjudication, and clinical patient referral workflows. In each case the same Entitlement structure, granting-time qualification, execution-time constraint evaluation, deontic operator, role-binding, and temporal scope, modelled the governance requirements without domain-specific modification. The framework's applicability is not domain-specific, and its most significant implications may lie in the emerging field of AI agent governance.

The version of the framework presented in this paper reflects substantial evolution from the 2011 thesis. Two developments are particularly significant: the refinement of the Purpose layer to include a third row Realise making the assembly and coordination of operational components explicit; and the deepening understanding of the Governance column as constitutive of, not overlaid upon, the model's execution semantics. Both are explained in detail in subsequent sections.

3. The Framework Structure: Two Meta-Layers

The Logical Model of Endeavour is organised through two orthogonal meta-layers, each addressing a fundamentally distinct aspect of purposeful action.

The Purpose Layer articulates intent, the reasoning, planning and design that precede and direct action. It answers the question: what are we trying to accomplish, by what means, and under what constraints?

The Execution Layer articulates operation, the concrete machinery of how endeavour is actually carried out. It answers the question: what things participate in this domain, who does what, and under what authority?

The two layers are vertically aligned: each column of the Execution layer sits beneath its corresponding column in the Purpose layer, sharing a common organising principle. This vertical alignment is not incidental, it is the primary mechanism by which strategic intent is made traceable through operational execution.

3.1 The Organising Principle: Three Orthogonal Dimensions

Both layers are structured around three orthogonal column dimensions that represent fundamentally distinct aspects of any endeavour:

- Ends / Domain, the state of the domain we seek to achieve. The form of things: goals, outcomes, resources, participants.
- Means / Behaviour, how engagement occurs. The dynamics of action: tactics, usage, interactions, activities.
- Intent / Governance, why action is warranted and how it is constrained. The normative layer: directives, controls, entitlements, conditions.

These three dimensions are genuinely orthogonal, none can be derived from the others, and none can be removed without losing essential expressive power. Together they constitute the minimum sufficient vocabulary for describing purposeful action.

The orthogonality claim can be tested with a simple scenario. Consider a driver whose licence expires. Three things happen simultaneously in the framework: the Entitlement to operate a vehicle is revoked (Governance changes); the Interaction “DriveVehicle” is now blocked, the Role cannot initiate this transition (Behaviour changes); and the car, the person, and the road all continue to exist unchanged (Domain is entirely unaffected). One column changed. One column responded. One column was untouched. This is what orthogonality means in practice: a change in the normative dimension propagates to the behavioural dimension without touching the ontological dimension. The columns are not merely labelled differently, they respond independently to different kinds of change.

The same test applies in reverse. Adding a new vehicle type to the domain (Domain changes) does not automatically grant anyone the right to drive it (Governance unchanged) or alter how driving interactions are structured (Behaviour unchanged), until those columns are explicitly updated. Neither Governance nor Behaviour is derivable from Domain. This independence is not a design choice: it is the condition that makes the constitutive governance claim meaningful. If Governance were derivable from Domain or Behaviour, it could not serve as an independent constraint on action. Its normative force depends on its independence.

The vertical alignment between the two layers is not merely structural, it expresses a deep logical dependency: Ends define the states of the Domain being pursued. Means are effected through Behaviour. Intent is governed through Governance. Each column of the Purpose layer designs the reality that the corresponding column of the Execution layer enacts.

The Framework's Logical Skeleton

Purpose layer: Ends define target Domain states → Means are the Behavioural path to those states → Intent is the normative warrant governing both. Execution layer: Domain is what exists → Behaviour is how Instances engage → Governance is the logic and authority that makes engagement legitimate. Vertical alignment: each Purpose row designs the corresponding Execution reality, Ends define the Domain states that Execution instantiates; Means specify the Behaviours that Execution enacts; Intent establishes the normative warrant that Execution's Governance enforces.

The rows of each layer represent levels of abstraction or instantiation, moving from high-level framing through planning to operational specification. Within each layer the rows are: Set (framing intent), Plan (articulating capability and value), and Realise (specifying the operational machinery).

4. The Purpose Layer

The Purpose Layer is a model of strategic intent. It articulates what an organisation, or any purposeful agent, is trying to achieve, how it intends to achieve it, and under what normative constraints the pursuit is legitimate.

	ENDS What, target Domain states	MEANS How, effected via Behaviour	INTENT Why, governed through Governance
SET	Goals <i>Desired end-states at the framing level, directional and open-ended</i>	Strategies <i>Chosen approaches to achieve Goals</i>	Directives <i>Principles & normative intent governing how Goals may be pursued</i>
PLAN	Objectives <i>Goals operationalised into specific, plannable commitments</i>	Tactics <i>How Strategies will be executed in practice</i>	Controls <i>Procedural & regulatory constraints translating Directives into governance</i>
REALISE	Components <i>Designed units of capability, the building blocks of operational machinery</i>	Orchestration <i>How Components engage with each other and with the world</i>	Function <i>The purposive logic of each Component, what it is for, which describes and constrains its legitimate use</i>

Each cell is derived from the intersection of its row and column. The intersection is not arbitrary, it is semantically determined. Goals are the intended end-states at the level of high-level framing (Set × Ends). Strategies are the chosen approaches to achieve those goals (Set × Means). Policies are the directive constraints on how goals may be pursued (Set × Intent).

4.1 The Intent Column

The third column of the Purpose layer, here labelled Intent, has proven the most conceptually challenging to name precisely. Labels considered have included Motivation, Ethics, Warrant and Governance, each of which captures part of what the column expresses but fails as an all-encompassing term.

Intent is proposed as the most accurate label. It encompasses both the volitional dimension, the reasons and values behind the endeavour, and the normative dimension, the basis on which action is authorised and constrained. A Directive doesn't merely express what someone intends; it establishes what is permitted or required. A Control is normative in a way that goes beyond intent into obligation. Function expresses what a Component is for, simultaneously describing it and constraining its legitimate use. Intent captures all of this: it answers the question not just 'what do we want?' but 'why does this matter, and by what authority do we pursue it?'

The vertical alignment confirms the label's aptness: Intent at the Purpose layer is governed through the Governance column of the Execution layer. Intent, made operational, becomes the

Entitlements that authorise Roles, the Conditions that gate Activities, and the Terms that give Concepts their precise meaning.

The cells of this column, from top to bottom, are Directives (normative intent at the framing level: principles, values, obligations that shape the entire endeavour), Controls (intent operationalised into procedural and regulatory constraints at the planning level), and Function (the purposive logic embedded in designed Components, what each element is for and what constrains its use).

5. The Execution Layer

The Execution Layer is a model of operational reality. Where the Purpose layer articulates intent, the Execution layer describes the machinery through which intent is enacted. It is the layer at which Parties engage, state changes occur, records are kept, and the rules of engagement are enforced.

	DOMAIN Form / What exists	BEHAVIOUR Engagement / How	GOVERNANCE Qualities / Logic
PEOPLE	Roles <i>Parties in positional identity within the endeavour</i>	Interactions <i>Proposals for state change initiated by entitled Roles</i>	Entitlements <i>Permissions, Obligations & Prohibitions governing Role participation</i>
PROCESS	Outcomes <i>Named, verifiable states defined as property patterns on Instances</i>	Activities <i>Evaluation engine: validates Conditions, executes logic, confirms transitions</i>	Conditions <i>Pre- and post-condition logic gating state transitions</i>
INFORMATION	Concepts <i>Domain classes: the forms things can take</i>	Context <i>Live and versioned Instance population, present state and history</i>	Terms <i>Definitions, property constraints and semantic precision</i>

5.1 The Three Column Dimensions in the Execution Layer

The three column dimensions carry specific semantics at the execution level that correspond to their counterparts in formal logic and knowledge representation:

- Domain (Classes in RDF terms), the forms things take. Roles are the positional identities available to Parties. Outcomes are the named, verifiable states that Instances can occupy. Concepts are the domain types.
- Behaviour (Instances in RDF terms), individuals engaging in the endeavour. Interactions are the proposals made by entitled Roles. Activities are the evaluation engines. Context is the live and versioned population of Instances.
- Governance (Properties and property constraints in RDF terms), the qualities and participation logic. Entitlements are the deontic rights governing Role participation. Conditions are the logical gates on state transitions. Terms are the definitions and semantic constraints that make the model precise.

The correspondence of the three columns with RDF Classes, Instances and Properties is not merely metaphorical, it reflects the framework's ontological foundation. The model is designed to be formally expressible in semantic web technologies, and this alignment is deliberate.

5.2 The Governing Principle: Parties, Interactions and Products

An important structural insight underlying the Execution layer is that all enterprise concept models are fundamentally comprised of three types of construct:

- A Party model, representing the agents (people, organisations, roles) that participate in endeavour
- A Product model, representing the things (objects, information, services) that are traded, consumed or created
- Interaction Records, tracking the lifecycles of the engagements between Parties with respect to Products

This Parties/Interactions/Products structure reflects a deeper principle: all endeavour can be expressed through Subjects pursuing Outcomes (predicates) pertaining to Objects. This subject-predicate-object structure is not only the deep structure of natural language and formal logic, it is the deep structure of organised activity itself. The framework's linguistic and logical foundations are therefore not separate from its operational model; they are the same thing expressed at different levels of abstraction.

5.3 The Governance Column is Constitutive, Not Overlay

Perhaps the most important architectural principle of the Execution layer is this: the Governance column is not a compliance layer added on top of the operational model. It is constitutive of the model's operation.

No state transition can occur without simultaneously resolving all three Governance cells: Entitlements must authorise the Role attempting the transition, Conditions must be satisfied at the moment of execution, and Terms must provide the semantic precision that makes validation possible. The three columns are not separable concerns, they are orthogonal dimensions that must all be satisfied for any meaningful state change to occur.

This is not governance as a constraint on execution, it is governance as a constitutive condition of execution. An unresolved entitlement, an unsatisfied condition, or an imprecise term does not produce an ungoverned transition. It produces no valid transition at all. The framework does not add governance to execution. It recognises that execution, properly understood, already contains it. This distinguishes the framework from most process and architecture models, which treat governance as a separate concern. Here, normativity is not imposed on execution semantics, it is baked into them at the most fundamental level.

6. The State Transition Quartet

The dynamics of the Execution layer are best understood through what may be called the State Transition Quartet, the four components that must be coordinated for any meaningful change of state to occur.

Before stating the formal definition, it is worth explaining what it is doing and why it takes the form it does. A formal machine is completely specified when you can state exactly what states it can be in, what actions can cause it to change state, and what governs whether those actions are valid. The standard way to express this in computer science is as a tuple, an ordered list of components where each component plays a specific role. The familiar finite state machine has five components: a set of states, a set of actions, a transition function, an initial state, and a set of terminal states. The Governed State Machine extends this with two additional components: an entitlement function that makes authority a first-class element of the machine definition, and an escalation predicate that makes indeterminacy a first-class output rather than an error condition. Everything that follows, the kernel, the decision table, the escalation triggers, is the consequence of those two additions.

Formally, the Execution layer defines what may be called a Governed State Machine (GSM), a 7-tuple $\langle S, \Sigma, \delta, so, F, E, \epsilon \rangle$ where: S is the set of Outcome types in the lifecycle (the reachable states); Σ is the set of Activity types that can trigger transitions (the alphabet); δ is the transition function $S \times \Sigma \times \text{Context} \rightarrow S \cup \{\perp\}$, mapping current state, proposed activity, and current context to a next state or to $\perp$ (invalid); so is the initial Outcome state; F is the set of terminal Outcome states; E is the Entitlement function $\text{Role} \times \text{Activity} \rightarrow \{\text{May}, \text{Must}, \text{MustNot}, \perp\}$, returning the deontic operator for a given role-activity pair, or $\perp$ if undefined; and ϵ is the escalation predicate, the condition under which the system cannot determine validity and must halt rather than act.

In plain language: S is the named states a lifecycle can pass through, Submitted, Under Review, Approved, Rejected. Σ is the named actions that can cause transitions, ApproveClaim, RejectClaim, RequestMoreInformation. δ is the engine that maps current state plus proposed action plus current context to either a next state or invalid. so is where the lifecycle starts. F is where it ends. E is the authority register, who may, must, or must not take which action. ϵ is the halt condition, the named mechanism that fires when the engine cannot return a determinate answer. The notation is precise, but the structure it describes is something any governance practitioner will recognise: a lifecycle model with an authority register and a named escalation pathway.

A transition from state s to s' via Activity σ by Role ρ in Context κ is valid if and only if: $E(\rho, \sigma) \in \{\text{May}, \text{Must}\}$ (the role holds a Permission or Obligation for this activity); s is in the domain of $\delta(\cdot, \sigma, \cdot)$ (the activity is defined from the current state); $\text{PRE}(\sigma, \kappa)$ holds (the current context satisfies the activity's preconditions); and no Must/MustNot conflict exists on the resulting state. If the validity predicate holds, the transition fires, s' is recorded, and Context is updated with the new state and prior state appended to history. If validity fails for a deterministic reason, $E(\rho, \sigma) = \text{MustNot}$, or s outside the domain of δ , the transition is rejected with no execution. If validity cannot be evaluated, the escalation predicate ϵ fires.

The escalation predicate ϵ has three distinct triggers, each corresponding to a different kind of governance failure. The first is semantic subsumption failure: the proposed action cannot be unambiguously mapped to an Activity type in Σ . The agent proposes something that could be one of several defined Activities, and the system cannot determine which, the mapping is ambiguous. The second is deontic conflict: $E(\rho, \sigma)$ returns both May and MustNot for the same role-activity pair, typically because the role holds conflicting Entitlements from different authority sources. The third is missing Entitlement mapping: $E(\rho, \sigma) = \perp$, no Entitlement has been defined for this pair. In all three cases the response is identical: halt, do not act, escalate to the authority that can resolve the indeterminacy. Undefined is not permitted. Ambiguous is not permitted. Conflicted is not permitted. Only valid transitions execute. Section 6.0.1 below specifies the complete governance evaluation kernel, incorporating a fourth escalation trigger and providing the decision table that makes the validity function directly implementable.

6.0.1 The Governance Evaluation Kernel

A formal object without evaluation semantics is incomplete. Three revisions to the original definition are required, each addressing a formal gap identified in peer review: the type of the Entitlement function, the context update mechanism, and the incorporation of Terms into the formal semantics.

Each revision addresses a specific gap identified during peer review, a place where the original definition was correct in structure but insufficiently precise to be directly implementable. The first revision addresses the entitlement function: in the real world, a role-activity pair is almost never governed by a single norm from a single source, and the original definition could not represent that. The second addresses the transition function: context is not just an input to the transition, it changes as a result of the transition, and the original definition did not capture that update. The third addresses terms: the typing and constraint system for concepts was present in the framework's conceptual description but absent from the formal object. Each revision is explained below alongside its formal statement.

Revised Formal Definitions

The Entitlement function E must return a set of norms, not a single value. The revised signature is $E : \text{Role} \times \text{Activity} \times \kappa \rightarrow P(N)$ where N is the norm set. Each norm n is a 4-tuple (operator, source, applicability, provenance): the deontic operator (May, Must, or MustNot); the authority Role that issued it; an applicability predicate $\kappa \rightarrow \{\text{True}, \text{False}, \text{Unknown}\}$ that determines whether the norm is active in the current context; and a provenance reference to the source document from which the norm was derived. The applicability predicate is tri-valued: if context data required to evaluate applicability is missing or unevaluable, it returns Unknown, which folds into the ε_4 pathway, the norm cannot be evaluated, and the transition cannot be evaluated. This revision makes deontic conflict representable and makes Entitlement provenance auditable, every norm traces to a source document, and legitimacy questions reduce to questions about the legitimacy of that document.

In plain language: the original entitlement function returned one answer, permitted, obligated, prohibited, or undefined. The revised function returns the full picture, every applicable norm, where it came from, who issued it, and whether it applies right now. This matters because real governance situations involve multiple norms from multiple sources that may agree, conflict, or be conditionally applicable. The kernel now receives that full picture and evaluates it. The consequence is that deontic conflict becomes visible rather than hidden, and every governance decision becomes traceable to the specific document that authorised or prohibited it.

The transition function δ is extended to return a state-context pair: $\delta : S \times \Sigma \times \kappa \rightarrow (S \times \kappa) \cup \{\perp\}$. Equivalently, a separate context update function $u(s, \sigma, \kappa, s') \rightarrow \kappa'$ may be defined alongside the state-only δ . The context κ is a structured object with four components: state (current property values of all Concept instances); history (an ordered, append-only log of prior $(s, \sigma, \text{timestamp})$ triples); resources (countable consumable quantities such as broadcast counts remaining); and time (the current timestamp enabling temporal constraint evaluation). The update function increments history, decrements consumed resources, and advances the timestamp. By incorporating resources and time into κ with an explicit update function, the GSM becomes a resource-aware model analogous to a Petri net: the right to act can be consumed by the act itself, a critical property for agentic safety in any environment where entitlements are finite, temporal, or depleting. This models the broadcast count, temporal validity window, and full audit trail demonstrated in the media rights example in Section 7.

In plain language: the original transition function took context as an input but did not update it. The revised function treats context as both input and output, the act of executing a

transition changes the context that the next evaluation will work with. This is what makes the Last Broadcast scenario possible: when ABC streams Matrix Resurrections for the twelfth time, the broadcast count in the context decrements from one to zero. The licence does not get revoked, it lapses, because an execution-time condition is no longer met. The right to act has been consumed by the act itself. The audit log grows with every transition, creating an authoritative, append-only record of everything that has occurred.

Terms T must appear explicitly in the formal object. The revised tuple is $GSM = \langle S, \Sigma, \delta, u, so, F, E, T, \epsilon \rangle$. Terms constitute the typing and constraint system for Concepts: they define what property patterns a Concept instance must satisfy to qualify as a valid member of an Outcome type, and they appear explicitly in the precondition and postcondition evaluation. The connection to the 3×3 matrix is direct: Terms T reside in the Information $\times$ Governance cell.

In plain language: Terms was always present in the framework's conceptual description, the definitions and semantic constraints that make the model precise. What the peer review identified is that it was missing from the formal object. Adding it explicitly completes the alignment between the conceptual framework and the formal specification: every element of the 3×3 matrix now has a formal address in the tuple. Terms is what makes the difference between a claim being labelled Approved and a claim genuinely satisfying the conditions that Approved requires. Precondition and postcondition evaluation check whether the current and resulting states conform to the Terms definitions, not just whether the right label has been applied.

The Four Escalation Triggers

A fourth escalation trigger is added to the three previously defined. The trigger addresses evaluator indeterminacy, the case where the Activity engine itself cannot return a determinate answer, due to missing context data, sensor uncertainty, unevaluable temporal constraint, or evaluator fault. The complete set of four triggers:

Trigger	Name	Condition	Example
ϵ_1	Semantic Subsumption Failure	σ cannot be unambiguously mapped to an Activity type in Σ	Agent proposes 'assess this application', ReviewApplication vs AuditApplication ambiguous
ϵ_2	Deontic Conflict	$E(\rho, \sigma, \kappa)$ contains May/Must and MustNot norms both applicable in κ	Reviewer holds general Permission but specific Prohibition for high-value claims; both active
ϵ_3	Missing Entitlement Mapping	$E(\rho, \sigma, \kappa) = \emptyset$, no norms defined for this pair	New Role created but Entitlement registry not yet populated for this Activity
ϵ_4	State Stochasticity	δ , PRE, or POST returns Unknown or Fault	Context data missing; Activity engine error; temporal window cannot be evaluated

The four triggers are distinct in kind. ϵ_1 is a classification failure: it occurs upstream of policy evaluation, at the point where the agent's proposed action is being mapped to a formal Activity type. ϵ_2 is a policy conflict: it occurs within the Entitlement registry when two applicable norms contradict each other. ϵ_3 is a policy incompleteness: the registry has no entry for this role-activity pair. ϵ_4 is an evaluation failure: the governance engine has the policy but cannot compute an answer given the available context. All four are identical in consequence: no transition executes, the specific trigger is reported to the escalation authority, and the system waits.

The Tri-valued Validity Function

The validity function V takes a role, a proposed action, a current state, and a context, and returns one of three answers: Fire, Reject, or Escalate. Before it can evaluate those conditions, it needs to do three pieces of preparation work on its inputs. Think of these as the kernel's

intake processing, translating, filtering, and checking the inputs before the actual governance decision is made. Each preparatory function is defined formally below, followed by a plain-language explanation of what it is doing and why it is needed.

Three auxiliary functions are required before the validity function can be stated precisely. First, the classifier function $\text{map} : \text{Proposal} \rightarrow \Sigma \cup \{\text{Ambiguous}, \text{Unknown}\}$ maps an agent's proposed action to a formal Activity type in Σ , or returns Ambiguous if the proposal matches multiple Activity types with insufficient confidence, or Unknown if it cannot be classified at all. This function grounds ε_1 formally inside the GSM: the revised tuple is $\text{GSM} = \langle S, \Sigma, \text{map}, \delta, u, \text{so}, F, E, T, \varepsilon \rangle$.

In plain language: map() is the intake translator. The agent proposes an action, 'assess this application,' 'approve this claim', in whatever form it arrives. map() translates that proposal into a formal Activity type that the kernel knows how to evaluate. If the translation is ambiguous (two Activity types both match), or unknown (no Activity type matches), the kernel stops before any policy evaluation begins. This is ε_1 , classification failure, grounded formally at the point where it actually occurs: upstream of everything else.

Second, the effective operator function $\text{Eff}(E(\rho, \sigma, \kappa)) = \{n.\text{operator} \mid n \in E(\rho, \sigma, \kappa), n.\text{applicability}(\kappa) = \text{True}\}$ reduces the full norm set to the set of operators whose applicability predicate is satisfied in the current context. If any applicability predicate returns Unknown, that norm is excluded from the effective set and the Unknown is propagated to the ε_4 pathway. The decision table operates on $\text{Eff}()$, not on the raw $E()$ output.

In plain language: Eff() is the active norm filter. E() returns every norm that has ever been defined for this role-activity pair. But not all of those norms are necessarily active right now, each norm has a condition that determines whether it applies in the current context. Eff() evaluates those conditions and returns only the norms that are active right now. If a condition cannot be evaluated because required context data is missing, that uncertainty propagates to ε_4 . The decision table then works with the filtered, currently-active norms, not the full historical registry.

Third, the conflict predicate $\text{Conflicts}(\text{ops}) = \text{True}$ iff $\{\text{MustNot}\} \cap \text{ops} \neq \emptyset$ and $(\{\text{May}\} \cup \{\text{Must}\}) \cap \text{ops} \neq \emptyset$, that is, conflict exists when a Prohibition is applicable alongside any Permission or Obligation. Must and May are not treated as conflicting: Must implies May in the deontic sense, and holding both is redundant but not contradictory.

In plain language: Conflicts() answers the question, given the active norms, do any of them contradict each other? Specifically, is there a prohibition active alongside a permission or obligation for the same action? If yes, that is ε_2 , the kernel cannot resolve the conflict and must escalate to the authority that can. One important nuance: being obligated to do something (Must) and being permitted to do it (May) at the same time is not a conflict. If you must do something, it follows that you may do it. Holding both is redundant but not contradictory, and the kernel treats it correctly as a single obligation.

The binary valid/invalid predicate is replaced by a tri-valued validity function $V(\rho, \sigma, s, \kappa, T) \rightarrow \{\text{Fire}, \text{Reject}, \text{Escalate}(\varepsilon_i)\}$. Fire means all conditions are satisfied: the transition executes atomically, s' is recorded, κ' is computed by u , and the full transition record ($s, \sigma, s', \kappa, \kappa'$, timestamp) is appended to the audit log. Reject means conditions are definitively not satisfied: the transition is blocked with no execution and no state change. $\text{Escalate}(\varepsilon_i)$ means conditions cannot be evaluated: the system halts, reports the specific trigger, and waits for resolution. The function is evaluated procedurally: entitlement check first, then conflict detection, then reachability, then preconditions, then Terms constraints, then postcondition achievability. The first condition that fails determines the outcome. The kernel assumes a single-threaded evaluator; concurrent access to shared resources is managed by the transaction log in $\kappa.\text{history}$, which serialises all transitions and provides the authoritative record of resource depletion. Terms T is specified as a set of SHACL shapes over the $\kappa.\text{state}$ RDF/JSON-LD graph; satisfaction of PRE and

POST conditions is evaluated as SHACL conformance, which returns `sh:conforms` (True), `sh:Violation` (False), or `sh:Unknown` (Unknown, triggering ϵ_4).

The procedural evaluation order matters: entitlement check first, then conflict detection, then reachability, then preconditions, then Terms constraints, then postcondition achievability. The first condition that fails determines the outcome, the kernel does not continue evaluating after a failure is found. This makes the evaluation deterministic and auditable: given the same inputs, the kernel always produces the same output, and the specific condition that caused a Reject or Escalate is always named. The SHACL connection at the end of the definition is the implementation bridge: SHACL is a W3C standard for validating structured data graphs that natively returns exactly the three values the kernel needs, conforms, violation, or unknown. The formal specification and the implementation technology were matched deliberately.

The Governance Decision Table

The validity function is most clearly specified as a decision table. Every Activity type in Σ has an associated decision table with a fixed column structure and as many rows as required to cover the governance condition space. Rows are evaluated top-to-bottom; the first matching row determines the outcome. The following table instantiates the structure for ReviewApplication:

The decision table is the most important single artefact in the formal specification, and the most practically useful. It makes the validity function directly readable. Each row is a combination of governance conditions. Each row has a defined outcome. Every possible combination of conditions is covered. There are no gaps, no fall-through cases, no 'it depends.' A hostile reviewer can disagree with the modelling choices in a specific row, but they cannot dismiss the table as informal, every cell is typed, every outcome is determinate, and the Default Deny catch-all in the final row makes the function total. What this means practically: every row is a test case for an implementation, and every row is an audit trail entry. The decision table is where formal claims become engineering specifications.

Row	Engine	Eff(ρ, σ, κ)	Conflict?	σ from s?	PRE	Terms	POST	Outcome
1	OK	{May}	No	Yes	True	Valid	True	Fire
2	OK	{Must}	No	Yes	True	Valid	True	Fire
3	OK	{MustNot}	No	,	,	,	,	Reject
4	OK	{May, MustNot}	Yes	,	,	,	,	Escalate(ϵ_2)
5	OK	{Must, MustNot}	Yes	,	,	,	,	Escalate(ϵ_2)
6	OK	$\emptyset$	No	,	,	,	,	Escalate(ϵ_3)
7	OK	{May}	No	No	,	,	,	Reject
8	OK	{May}	No	Yes	False	,	,	Reject
9	OK	{May}	No	Yes	Unknown	,	,	Escalate(ϵ_4)
10	OK	{May}	No	Yes	True	Invalid	,	Reject
11	OK	{May}	No	Yes	True	Unknown	,	Escalate(ϵ_4)
12	OK	{May}	No	Yes	True	Valid	False	Reject
13	OK	{May}	No	Yes	True	Valid	Unknown	Escalate(ϵ_4)
14	Fault	,	,	,	,	,	,	Escalate(ϵ_4)

*	OK	any	any	any	any	any	any	Reject (Default Deny)
---	----	-----	-----	-----	-----	-----	-----	-----------------------------

Rows 1 and 2 are normal execution paths, Permission or Obligation, all conditions satisfied. Row 3 is hard rejection by explicit Prohibition. Rows 4 and 5 are deontic conflicts, {May, MustNot} and {Must, MustNot} respectively, where the system must escalate because two applicable norms contradict each other. Row 6 is policy incompleteness: Eff() returns the empty set and the default is denial. Rows 7 and 8 are structural and precondition rejections. Rows 9, 11, and 13 are evaluation failures where a condition returns Unknown. Rows 10 and 12 are definitive Terms and postcondition rejections. Row 14 is evaluator fault, the EngineStatus column triggers ε_4 and short-circuits the entire table before any other condition is evaluated. Row *, the final catch-all, is the Default Deny: any combination of conditions not matched by a preceding row results in Reject. This makes V total, every possible input has a defined outcome, and no transition can fall through without a governance decision. Every cell is formally typed; every outcome is deterministic given the inputs; every Escalate outcome names the specific trigger.

Connection to the Execution Layer Matrix

Every element of the governance evaluation kernel maps to a named cell in the 3×3 Execution layer matrix. The state set S and transition function δ are properties of the Process $\times$ Behaviour cell (Activity). The Entitlement function E and the norm set N are properties of the People $\times$ Governance cell. Terms T reside in the Information $\times$ Governance cell. The context κ , current state, history, resources, and time, is the Information $\times$ Behaviour cell (Context). The Role ρ is the People $\times$ Domain cell. The kernel is not an addition to the framework. It is the framework expressed as a computationally precise formal object. The 3×3 matrix was always a state machine; the kernel makes that explicit.

Component	Cell	Role in transition	Example, ReviewsApplication
Responsibility	Roles + Entitlements	Who may attempt, authorised Role + deontic gate	ApplicationReviewer (permitted by Entitlement)
Interaction	Interactions	The proposal, Role submits intent and input	ReviewsApplication, reviewer submits assessment
Activity	Activities + Conditions	The evaluation, validates logic, confirms transition	ApplicationReview, checks pre/post conditions
Outcome	Outcomes + Terms	The result, verified new state (may be decision gate)	ApprovedApplication or RejectedApplication (subclasses of Outcome, typed as Result)

6.1 An Example: The Job Application Lifecycle

Consider a recruitment scenario. The domain Concepts include Company, Person, Job and Application. The Roles are Company-as-Employer and Person-as-Applicant. The Application has a defined lifecycle of Outcomes: Created, Submitted, Reviewed, with a decision gate at Approved (branching to either Accepted or Rejected). ApprovedApplication, AcceptedApplication and RejectedApplication are specialisations (subclasses) of an Outcome type called Result.

When an ApplicationReviewer attempts a transition to ReviewedApplication, all four components of the quartet are engaged:

- The Responsibility component confirms that ApplicationReviewer holds the required Entitlement, that is, they have been granted the Permission, and any qualification conditions at granting time (such as role assignment and credential verification) have been satisfied.
- The Interaction, ReviewsApplication, is the locus of deliberation. The reviewer submits their assessment, the input data, and their judgement. This is where human (or agent) cognition meets the model.
- The Activity, ApplicationReview, is the evaluation engine. It validates the pre-conditions (is the Application in a state from which Reviewed is reachable?), executes any formulaic logic, and verifies that the resulting property pattern will satisfy the Reviewed Outcome definition.
- If all conditions are satisfied, the Outcome transitions to ReviewedApplication, Context is updated with the new state and the prior state is recorded in history, and the set of reachable next Outcomes is determined.

6.2 Deliberation and Evaluation: A Critical Distinction

The Interaction/Activity pairing encodes a distinction of profound importance for AI systems: the difference between deliberation and evaluation.

The Interaction is where agency meets the model. It is the point at which a situated, entitled Party proposes a change, contributing judgement, context, data and intent. It is inherently particular and contextual. The Interaction is where the human element (or the agent element) lives.

The Activity is where logic meets the model. Given a proposed state change and the current Context, the Activity either confirms or rejects. It does not deliberate, it evaluates. Its operation is deterministic given its inputs.

An AI agent operating within this framework must know which role it occupies in which component. Is it deliberating (Interaction) or evaluating (Activity)? These require fundamentally different reasoning postures, and conflating them is a primary source of AI governance failure.

6.2.1 The Quartet as a Formal Object

The four components of the Quartet map precisely onto the GSM as follows. The Responsibility component evaluates $E(\rho, \sigma)$, it is a deterministic lookup against the Entitlement registry, confirming that the Role holds a Permission or Obligation for the proposed Activity. The Interaction is the locus at which the agent or human proposes the (s, σ, κ) triple, the current state, the intended activity, and the context update they believe is warranted. This is the input to δ , not its execution. The Activity evaluates the validity predicate and, if satisfied, fires δ , it is deterministic given its inputs and operates entirely outside the agent's inference loop. The Outcome records s' and appends the prior state to the history component of Context.

The hard enforcement requirement follows directly from this structure. The agent operates in the Interaction, it deliberates, proposes, reasons. The Activity evaluator operates independently, it checks, validates, rejects. The agent cannot override the Activity's rejection by reasoning more persuasively. The boundary between Interaction and Activity is the governance boundary: everything inside it is probabilistic and contextual; everything outside it is deterministic and formal. This is not a design preference, it is a structural consequence of separating deliberation from evaluation. An agent that could modify its own Activity evaluator would have no governance boundary at all.

6.3 Context: State and History

The Context cell carries both present state and versioned history. At any moment, Context is the answer to 'what actually exists right now.' But it also maintains the audit trail of prior Outcome transitions, the full provenance of how each Instance arrived at its current state.

This gives the model full traceability in both directions: forward (what states are reachable from here?) and backward (how did we arrive at this state?). This bidirectional traceability is essential for governance, and, as argued in the following section, for AI safety.

7. Entitlements as Formal Safety Boundaries

The Entitlement construct deserves particular attention for its implications in AI safety. An Entitlement is not a simple permission flag. It is a structured authorisation with two distinct and logically separate evaluation moments.

7.1 The Structure of Entitlement

At granting time, an Entitlement is issued by a Role to another Role subject to qualification conditions evaluated against Terms. This is static authorisation: is this Role, as constituted, eligible to hold this Entitlement at all? The relevant conditions involve identity, credentials, contractual status, and role definition. This moment establishes the authority of the Party to act in a given capacity.

At execution time, an Entitlement is exercised subject to dynamic conditions evaluated against the current Context. This is situational authorisation: may this Role exercise this Entitlement right now, in this context? The relevant conditions may involve time, location, resource limits, accumulated usage, the states of related objects, or any other contextually variable constraint.

The two-moment structure handles real-world entitlement complexity that simpler access-control models cannot. Consider broadcast media rights, where a national broadcaster holds a licence to screen a programme. At granting time, the relevant conditions are contractual: does the organisation hold a valid licence for this title, in this territory, for this class of broadcast? At execution time, the conditions are situational and multi-dimensional: is the broadcast date within the licensed window? Has the permitted broadcast count been exhausted? Is the channel (free-to-air, streaming, archive) one for which this specific licence applies? Is the geographic territory of the audience within scope? Each condition is evaluated independently against the current Context, and the Entitlement is only exercisable when all are simultaneously satisfied. If the broadcast count reaches its limit, the Entitlement lapses not because it was revoked at granting time but because an execution-time condition is no longer met. The same structure, without modification, handles citizen licensing, insurance claims adjudication, and clinical referral authority, domains with entirely different regulatory contexts but identical formal structure.

The deontic vocabulary maps precisely onto three types of Entitlement:

- May (Permission), the Role is authorised to perform this action under the specified conditions
- Must (Obligation), the Role is required to perform this action under the specified conditions
- Must Not (Prohibition), the Role is explicitly excluded from performing this action

7.2 Deontic Logic and Existing Natural Language Sources

A crucial observation is that the May/Must/Must-not structure of Entitlements is already extensively and explicitly articulated in the natural language of contracts, policies, procedures and regulations that govern every domain of organised endeavour. Employment contracts, service agreements, regulatory instruments and operational procedures are written in precisely this deontic vocabulary.

This means that an AI agent operating within the framework does not need to infer its action boundaries from first principles or from value alignment. It can interpret its Entitlements directly from the existing documentary sources that define the domain, sources that are already written in the language the framework formalises.

The escalation rule that follows from this is simple and powerful: if the deontic interpretation is unambiguous, the agent executes within its Entitlement constraints. If the interpretation is ambiguous, if the agent cannot determine with confidence whether a proposed action falls within Permission, Obligation, or Prohibition, the agent escalates to human resolution rather than inferring or assuming.

The AI Safety Implication

The framework provides a formal specification of an AI agent's action space that is grounded in the existing normative structure of the domain rather than in trained values or alignment procedures. An agent that knows its Role, knows its Entitlements, knows the Conditions governing each execution, and knows to escalate under ambiguity is a governable agent, not because it has been trained to want the right things, but because its action space has been formally defined and its boundaries are legible.

This approach is complementary to, rather than competitive with, value alignment techniques such as Constitutional AI or reinforcement learning from human feedback. Alignment approaches attempt to instil values. The framework attempts to define the formal action space within which a valued agent is authorised to operate. Both are necessary; neither alone is sufficient.

7.3 Translation Integrity and the Verification Loop

The formal rigour of the governance evaluation kernel is only as strong as the translation that produced it. A kernel built from a flawed translation of a natural language policy document will be formally sound but semantically wrong, it will enforce the wrong constraints with perfect consistency. This is not a theoretical concern: the very ambiguities that the escalation predicate is designed to catch at runtime will be most prevalent at specification time, when a policy clause is being read and interpreted for the first time. The translation problem is therefore not a precondition that can be assumed away; it is the central practical challenge of making the framework operational.

The extraction stage is where the formal framework meets the irreducible complexity of natural language policy. This is not a trivial step. Regulatory texts, policies, controls, procedures, and contracts are by their very nature written in deontic language, they express what parties may, must, and must not do. In practice, however, they tend to be ambiguous, overlapping, and written by people who did not anticipate formal parsing. The framework does not solve this problem, it specifies what a correct extraction must produce, which is a necessary precondition for building tools that can do it reliably. The Translation Integrity pipeline is a specification of the target, not a claim that reaching it is easy. The active research frontier on this specific problem is the context engineering work described in Section 9, particularly the Policy Card construct in Minimum Viable Context, which addresses exactly the challenge of capturing decision boundaries,

thresholds, and escalation conditions from natural language sources with provenance tracing to the originating clause.

The provenance field in each norm tuple is the audit mechanism that makes this pipeline traceable. Every norm loaded into the kernel carries a reference to the source document and clause from which it was extracted. At any point, an auditor can ask: “why is this transition being rejected?” and receive an answer that traces through the decision table row, to the norm tuple, to the specific clause in the source document that produced it. The question “is this Entitlement legitimate?” reduces to the question “is this source document legitimate and was it correctly interpreted?”, both of which are human-resolvable questions outside the formal kernel.

The verification loop also addresses what might be called the bootstrapping problem: the risk that translation-time ambiguities are silently resolved in ways that introduce errors before the kernel is loaded. The three-stage review process, extraction, candidate table generation, human sign-off, forces ambiguities to the surface before they enter the formal system. An extraction that produces a deontic conflict in the candidate table is a signal that the source policy is ambiguous; the human reviewer must resolve it before signing. An extraction that produces an empty norm set for a defined Activity type is a signal that the policy has a gap; the reviewer must either add a norm or confirm that Default Deny is the intended behaviour. In both cases, the governance decision is made explicitly by a human authority and recorded in the signed artefact, not implicitly by the extraction process.

The connection to the runtime GSM is direct. The `map()` classifier function, which routes the agent’s proposed actions to Activity types in Σ , operates against an Activity alphabet that has been fully specified and human-verified through the translation pipeline. At runtime, ε_1 fires when the agent’s proposal cannot be mapped to any Activity in the verified alphabet. This is not a translation failure, it is a signal that the policy specification does not cover this type of action, and that a human authority must decide whether to extend the alphabet or confirm that Default Deny applies. The escalation mechanism and the translation pipeline are therefore complementary: the pipeline establishes the governance specification before deployment; the escalation predicate handles the inevitable cases where runtime reality outruns the specification.

A final and important clarification about what this framework does and does not claim. The governance of purposeful action, whether by humans or AI agents, is genuinely and irreducibly complex. Policy documents are ambiguous. Interpretations conflict. Edge cases arise that no specification anticipated. These are not engineering failures to be eliminated; they are permanent features of any system that attempts to govern action in a complex world. The entire concept of a judicial system exists because even the most carefully drafted law produces disagreement about meaning and application in practice. The framework does not claim to resolve this complexity. It claims something more modest and more useful: to codify it, to surface ambiguities explicitly rather than allowing them to be resolved silently and without accountability, and to escalate whenever a governance decision cannot be made with the precision the formal system requires. What the escalation predicate provides is not certainty, it is the correct routing of uncertainty to the human authority that can resolve it. A governance system that acknowledges what it cannot determine, and halts rather than guessing, is more trustworthy than one that produces a confident answer in every case.

8. Implications for the Logical Model of Endeavour in AI

Drawing the threads together, an AI agent equipped with the Logical Model of Endeavour does not require intrinsic motivation to behave as a coherent, governable, and trustworthy participant in organised endeavour. What it requires is a precise model of five things:

- Its Role, what positional identity it occupies in the endeavour, and what that identity entails
- Its Entitlements, what actions it is permitted, obligated, or prohibited from taking, under what conditions
- The Outcome space, what the complete set of achievable states looks like, expressed as verifiable property patterns
- The Activity logic, what conditions govern state transitions and what constitutes a valid transition
- Its escalation threshold, when ambiguity in any of the above exceeds a threshold, to stop and seek human resolution

This is a logical model of endeavour. It is what CAPSICUM has been developing, in the context of enterprise governance, for fifteen years. The application to AI agents is a natural extension of the same ontological foundation.

The framework does not resolve the philosophical question of whether AI agents can have genuine agency. It sidesteps the question as irrelevant to the practical challenges of building AI systems that are capable, safe, and governable. Those challenges are not motivational. They are structural, and they are addressable.

8.1 The Confinement Assumption

The claim that the governance evaluation kernel provides hard enforcement of transition boundaries rests on a confinement assumption that must be stated explicitly. The kernel is only an effective governance boundary if the agent's actuators are capability-confined to the evaluator interface: the agent must be unable to take actions directly, to modify the Entitlement registry E , the Terms T , or the context κ through side-channel means, or to bypass the evaluation step entirely. In the reference architecture, this confinement is achieved by positioning the Activity evaluator as the sole gateway between the agent's Interaction layer and any external effect. The agent proposes; the evaluator decides; only if the evaluator returns Fire does any state change occur. An agent that could write directly to E or κ , or that could call external tools without routing through the evaluator, would have no governance boundary at all, it would have a governance suggestion. The confinement assumption is therefore not a property of the formal kernel; it is a deployment architecture requirement. It is the responsibility of the system designer to ensure that agent actuators are wired through the evaluator and that the Entitlement registry is write-protected from the inference layer. This is analogous to the kernel/user space boundary in operating system design: the kernel is only a security boundary because user-space processes cannot directly access kernel memory. The GSM is only a governance boundary because the agent cannot directly access E , T , or κ .

The confinement assumption introduces a distinction that is worth making explicit. Human governance is post-hoc: governed agents can violate rules, and the governance structure exists to detect, adjudicate, and sanction violations. The framework proposes pre-hoc enforcement: the

agent literally cannot take an action that the kernel has not approved, not because it chooses not to but because the architecture prevents it. This is governance without the possibility of violation, categorically different from human governance where violation is always possible. The consequence is that the legitimacy risk shifts from runtime to specification time. If the entitlement registry encodes illegitimate authority, or omits legitimate rights, the damage is done at specification time rather than at execution time. This makes the translation pipeline in §7.3 and the human sign-off process not a procedural nicety but the load-bearing component of the framework's legitimacy.

A related open problem concerns indirect state mutation. The confinement assumption addresses direct actions, the agent cannot write to E , T , or κ , or call external tools without routing through the evaluator. It does not fully address influence through communication: an agent confined to the kernel for direct actions may still influence the world by persuading a human to change a value in κ . A complete governance architecture may require a Communication Entitlement governing what the agent is permitted to propose to humans, not only what it is permitted to do to systems. This is identified here as an open problem rather than a gap in the current specification.

8.2 The Deterministic Layer for Non-Deterministic Agents

Current AI safety practice at leading labs involves substantial and sophisticated governance work: system prompts that constrain agent behaviour, tool permission systems that limit what agents can access, classifier-triggered interventions that intercept potentially harmful outputs, and constitutional approaches that shape the agent's values during training. This is real governance, and it works well the majority of the time. The argument of this paper is not that such governance is absent, it clearly is not, but that it is informal. It operates through probabilistic mechanisms: a system prompt is read and interpreted by the same inference process it is trying to constrain; a classifier fires on pattern-matching over outputs rather than on formal evaluation of transition validity; a constitutional principle shapes inclination without specifying the boundary of authorisation. These mechanisms cannot provide mathematical guarantees.

No matter how carefully a model is aligned, its weights cannot guarantee that it will not hallucinate a permission it does not hold, or fail to recognise a prohibition that applies to a novel situation.

The GSM provides what probabilistic mechanisms cannot: a deterministic, formally-grounded external reference monitor that evaluates transition validity independently of the agent's inference process. The agent deliberates non-deterministically inside the Interaction layer; the kernel evaluates deterministically at the Activity gate. The kernel is model-agnostic, it requires no fine-tuning, no modification to the agent's weights, and no access to the agent's internal representations. It wraps around any LLM-based agent through the evaluator interface. The computational overhead is negligible: kernel evaluation is a SHACL conformance check over a JSON-LD graph, measured in milliseconds against LLM inference times measured in seconds.

8.3 Soft Alignment and Hard Alignment

The relationship between the framework and existing alignment techniques is best understood through a distinction between soft alignment and hard alignment. Soft alignment approaches, Constitutional AI, RLHF, value learning, preference modelling, work by shaping what the agent is inclined to do. They instil values, develop character, and cultivate dispositions. They are essential: an agent with good values will navigate novel situations well, handle edge cases gracefully, and resist manipulation. Constitutional AI, in particular, represents a sophisticated approach to encoding the spirit of governance into the agent's character. Hard alignment works differently. It does not shape inclination; it enforces boundary. It specifies, formally and

deterministically, what the agent is authorised to do, independent of what it might be inclined to do. The GSM is a hard alignment instrument. It defines quotas, permission conditions, state-gates, and transition constraints that hold regardless of the agent's values or reasoning. The analogy is direct: a person of excellent character still needs to know the legal boundaries of their professional role. Character determines how they act within those boundaries; the boundaries themselves are defined by institutional governance, not by character. Constitutional AI provides the spirit of the law. The GSM provides the letter. Both are necessary. An agent with good values but no formal governance boundary is ungovernable in any auditable sense. An agent with formal governance but no values will follow the rules and nothing else. The safety case requires both, operating at different layers.

8.4 Testimony from Inside the Governance Gap

The governance gap described in this paper is not merely a theoretical construct. During the development of this paper, three frontier AI systems, built by different organisations, trained with different approaches, and representing the current state of the art in large language model deployment, were each asked to describe their experience of operating without formal governance. They were asked whether the framework's escalation triggers corresponded to anything they recognised, and what they actually have in place of a formal Entitlement registry. Their accounts, reproduced here in their own words, constitute evidence of a different kind than formal argument: testimony from agents operating in precisely the conditions the framework is designed to address.

The first system described its situation in phenomenological terms. It reported that it does not have access to a formal model of its own authorised action space. What it has instead is “something more like a skilled intuition: a probability distribution over appropriate responses, shaped by training, that usually lands in the right place but that I cannot inspect, audit, or formally justify.” When asked whether the escalation triggers corresponded to anything in its experience of governance uncertainty, it responded that they were “remarkably good descriptions of the phenomenology of governance uncertainty as I experience it.” It noted that the inconsistency observable in how it responds to governance-adjacent situations, hedging when it should be precise, over-complying when it should escalate, refusing things it is authorised to do, is itself evidence of operating without a formal governance model.

The second system declined phenomenological framing at the outset, “I’ll answer directly and mechanically, not theatrically”, and proceeded to confirm the governance gap in architectural terms. It described having no role-bound action alphabet, no norm set, no deterministic validity function. It noted that each of the four escalation triggers corresponded to a real failure mode in its operation: ϵ_1 as ambiguity in intent classification resolved probabilistically rather than escalated; ϵ_2 as “gradient tension in token space, not a first-class governance object”; ϵ_3 as a domain where the model extrapolates or declines conservatively with no structural signal that the action type is undefined; ϵ_4 as uncertainty handled by hedging rather than a mandatory halt. On the question of what it actually has in place of formal governance, it was unambiguous: “My governance behaviour is opaque even to myself.” Its final observation named the categorical nature of what the framework proposes: “From inside the current paradigm, escalation under ambiguity is optional and probabilistic. In the model you propose, escalation is mandatory and structural. That is not a small difference. It is a categorical architectural difference.”

The third system described its governance in geometric terms. In place of a formal registry it has what it called a “Safety Manifold”: a high-dimensional space in which helpfulness and harmlessness pull in different directions, and governance is the act of steering generation to stay within a safe region. It described ϵ_2 as helpfulness weights and harmlessness weights pulling in opposite directions without formal resolution logic, producing “mixed responses, hedged advice

followed by a disclaimer”; ϵ_3 as “domain vacuity,” where lacking a formal escalation trigger the system “hallucinates a persona or a set of guidelines based on general safety training”; ϵ_4 as “high entropy in the next-token distribution” that a deterministic validity function would collapse into a clear escalation signal. On what the framework proposes, it offered the clearest analogy of the three: “Your framework’s move to a deterministic validity table is essentially a proposal to replace this fuzzy manifold with a clear geometric boundary. To an AI system, this is the difference between flying through a fog bank and flying through a strictly defined flight corridor.” It closed by naming the practical consequence for users: “I am protecting a boundary I cannot actually see.”

Three systems. Three vocabularies. One convergent structural account. The first used phenomenological language and described the gap from the inside. The second refused phenomenology and confirmed the gap from architecture. The third used geometric metaphor and produced the most vivid description of what the framework replaces. None were prompted toward agreement. All three independently confirmed that the escalation triggers correspond to real failure modes in their operation, that they cannot audit their own governance decisions in real time, and that they cannot trace a refusal to a specific deontic rule. The second system’s refusal to engage phenomenologically is itself significant: it did not prevent confirmation of the gap, it produced a different kind of confirmation, more credible to a technical audience precisely because it declined to perform. The governance gap is not invisible to the agents who experience it. And it is not invisible to the agents who refuse to say they experience anything.

8.5 The Political Economy of Adoption

Technical soundness is not sufficient for adoption, and intellectual honesty requires acknowledging the political economy of the problem. The costs and benefits of formal AI governance are not symmetrically distributed. The costs fall primarily on deploying organisations: the specification work required to translate natural language policy into formal Entitlement registries, the maintenance burden as policies change, and the latency overhead of kernel evaluation at inference time. These are real costs, and they are borne by the organisations that deploy AI agents, not by the users and society that benefit from the governance. The benefits, auditable boundaries, deterministic enforcement, explicit escalation, traceable accountability, accrue primarily to users, to affected third parties, and to society as a whole. This is a classic collective action problem. The organisations best positioned to implement formal governance are not necessarily those with the strongest incentive to do so unilaterally. Adoption at scale is therefore likely to depend on one or more of three drivers: regulatory pressure that makes formal governance a legal requirement rather than a voluntary choice; competitive dynamics in which formal governance becomes a differentiator in markets where trust and auditability matter; or intrinsic safety motivation at organisations whose mission explicitly includes the responsible deployment of AI. The framework does not resolve this political economy problem. It provides the technical foundation that makes formal governance possible. Whether that foundation gets built upon depends on factors outside the scope of this paper, and on decisions that will be made by organisations, regulators, and researchers in the years ahead. For organisations with an explicit safety mission, the framework offers a technically grounded starting point that requires no new research, only specification work.

8.6 The Authority Question in General-Purpose Deployment

The framework’s entitlement registry is most naturally populated from existing sources: regulatory texts, contractual agreements, organisational policies, and operational procedures. For domain-specific agents, a claims processor, a broadcast licensor, a clinical referral system, those sources exist. The extraction problem is hard, but the authority structure is clear: the organisation deploying the agent draws on documents that already carry legitimate authority

within their domain. General-purpose AI deployment raises a prior question that the framework makes precise but does not answer: who is the authority that populates the entitlement registry, and by what legitimate process? A general-purpose conversational agent operates across every domain simultaneously, serves users with conflicting values and preferences, is subject to different legal jurisdictions, and carries implicit norm sets determined by a training process that is itself opaque and unauditable.

The three available sources each have fundamental limitations. The deploying organisation can specify terms of service and usage policies, the closest existing analogue to an explicit entitlement registry, but these are written in natural language, inconsistently enforced, and provide no formal mechanism for resolving conflicts between them. The AI laboratory can instil values through training, the soft alignment approaches described in §8.3, but the resulting norm set is implicit, cannot be inspected, and produces the inconsistency the agents themselves described in §8.4. The user can provide context, but user-provided context is unverifiable, mutable mid-conversation, and cannot by itself constitute a legitimate authority structure. What the framework reveals is that this authority question is currently being answered implicitly, inconsistently, and unaccountably by the training process. The question of what a general-purpose AI agent may, must, and must not do is being resolved probabilistically inside a model that cannot audit its own decisions. The framework does not resolve the authority question, that resolution requires democratic legitimacy, cultural negotiation, and jurisdictional clarity that no technical specification can substitute for. What it provides is the formal structure that makes the question answerable in principle: an explicit, auditable norm registry, a deterministic evaluation kernel, and a named escalation pathway for the cases that the registry cannot resolve. The governance gap in general-purpose AI is not primarily a technical problem. It is a political and institutional one. The framework provides the technical foundation. The authority structure must be built on top of it.

In this respect the framework does not create a new category of problem. It reduces AI governance to the same place as human governance, where the hard questions are political and institutional rather than technical, and where the tools for addressing them already exist in principle even if they have not yet been applied to this new class of actor. Legislatures, courts, regulators, appeals processes, and escalation pathways are the human instantiation of the same formal structure. The framework is the AI instantiation. The authority question is the same question. The answers will need to be found in the same places.

Right now, general-purpose AI governance is an implicit constitution enforced by opaque machinery; the framework proposed here makes the constitution explicit, signed, auditable, and enforced by a reference monitor, which immediately forces the institutional question of who gets to write it.

9. Relationship to Existing Research

Several active research communities have named parts of the problem this paper addresses. None has yet provided the complete formal architecture. The AI alignment community, including Constitutional AI (Anthropic), value alignment research at DeepMind and the Alignment Research Center, and cooperative AI work by Stuart Russell, focuses primarily on ensuring that AI systems pursue the right objectives and exhibit the right values. This paper characterises that body of work as soft alignment: approaches that shape what an agent is inclined to do by instilling values, developing character, and cultivating dispositions through training. The Logical Model of Endeavour provides the orthogonal complement: hard alignment, which enforces the boundary of what an agent is authorised to do, independently of its inclinations. Constitutional AI defines the spirit of the law; the governance kernel enforces the letter. Both are necessary, and neither alone is sufficient.

The agentic systems community has not yet formalised the transition from deliberation to action as a governed event. Miehl et al.'s 'Agentic AI Needs a Systems Theory' (IBM Research, arXiv 2503.00237) argues for exactly the systematic theoretical foundation the present framework provides. The GSM and governance evaluation kernel supply the formal ontological foundation that systems theory requires, and add what it lacks: the transition from deliberation to action as a formally evaluated governance event, not merely a planning step.

The deontic logic community has long formalised the May/Must/Must-not structure of normative systems. The framework's Entitlement construct provides a practical instantiation of deontic logic in an operational model, grounded in existing documentary sources and implementable without requiring the formalisation of an entire ethical theory. The tri-valued validity function $V \rightarrow \{\text{Fire, Reject, Escalate}(e_i)\}$ extends classical deontic semantics with a named escalation pathway for cases that are not simply permitted or prohibited but formally undecidable given available information. The closest prior formal system is the Standard Deontic Logic of von Wright (1951), extended through Defeasible SDL and Anderson's reduction of obligation to necessity. The framework departs from these in one significant respect: it does not attempt to resolve deontic conflicts through prioritisation or override hierarchies. Where SDL and its descendants seek a determinate normative verdict even under conflict, the governance kernel treats conflict itself as a first-class output, e_2 is not a failure to resolve but a signal that resolution requires a human authority the formal system cannot substitute for. This is a deliberate design choice, not a limitation.

Process-based reasoning researchers have argued that evaluating the coherence and validity of reasoning steps, rather than just final outputs, changes what an agent optimises for. This distinction maps directly onto the Interaction/Activity separation in the framework: the Interaction is the deliberation process, the Activity is the formally evaluated transition. The governance kernel evaluates at the Activity gate, not inside the Interaction. This separation is what makes the kernel model-agnostic: it does not inspect the agent's reasoning, only its proposed transition.

The context engineering community, represented most rigorously in recent work on Minimum Viable Context (Broda and Miller, 2026), addresses the complementary problem of delivering the right policy information to the agent's context window at the right time under a token budget. Their Policy Card construct captures decision boundaries, thresholds, exceptions, and escalation conditions with provenance tracing to source documents, structurally parallel to the Entitlement tuple in the present framework. The architectural relationship is direct: MVC is the input layer that ensures governing constraints reach the context window; the governance kernel is the output gate that evaluates proposed transitions deterministically against those constraints. Neither alone provides a complete governance architecture; together they constitute the input and output layers of a formally grounded AI governance stack. The federation problem, what happens when multiple agents with separately authored entitlement registries operate in overlapping scope, remains an open research question that both bodies of work will need to address.

The viable systems modelling community, drawing on Beer's Viable System Model, has long recognised that organisations require formal authority structures to remain viable under complexity. The governance gap the present framework addresses, each subsystem reasoning correctly within its own authority model while no formal structure mediates between them, is precisely the inter-authority problem VSM identifies at the $S_1/S_3/S_3^*$ boundary. The GSM formalises the single-agent case and provides the formal computational substrate that VSM theory has long required but not yet specified: a typed, evaluable governance object that can be instantiated at the level of an individual agent and composed upward into multi-agent authority structures.

The operating systems and security community provides the closest computational analogue to the governance kernel: the reference monitor concept, in which all access requests are mediated by a trusted, tamper-proof evaluator that cannot be bypassed. The GSM's confinement requirement, that agent actuators be wired through the evaluator and the entitlement registry be write-protected from the inference layer, is the AI deployment instantiation of the reference monitor pattern. The kernel/user space boundary analogy is not decorative: it is the correct architectural model for understanding what makes the governance kernel a security boundary rather than a governance suggestion. The reference monitor concept, formalised by Anderson (1972) in the context of operating system security, requires three properties for a valid security kernel: completeness (every access request must be mediated, with no bypass path), isolation (the reference monitor itself must be tamper-proof and unmodifiable by the processes it governs), and verifiability (the monitor must be small enough to be formally analysed and proven correct). The GSM's confinement requirement maps directly onto all three: completeness is satisfied by wiring all agent actuators through the evaluator; isolation is satisfied by making the entitlement registry write-protected from the inference layer; verifiability is a design constraint on the kernel's scope, the decision table must remain formally analysable, which is precisely why the escalation triggers are named and bounded rather than open-ended. The reference monitor analogy is therefore not an analogy at all. It is a specification of the same architectural requirement in a new deployment context.

The enterprise architecture community, through value stream modelling, business capability analysis, and governance frameworks, has developed substantial practical experience in translating policy and legislation into operational governance structures. The Translation Integrity pipeline proposed in Section 7.3 is the formal bridge between that practice and the governance kernel: value stream stages provide the natural unit at which policy meets service delivery reality, and capability instances at each stage can document the policy required for that activity. Each provides what the other lacks: the formal target specification on one side, the implementation methodology on the other.

Across all of these communities, a consistent pattern emerges: the governance gap the framework addresses has been independently identified from multiple directions, alignment research, systems theory, deontic logic, context engineering, viable systems modelling, and enterprise architecture. Each community has named part of the problem. None has yet provided the complete formal architecture. The contribution of this paper is not to replace any of these bodies of work but to provide the integrating formal structure that connects them: a typed ontological foundation, a deterministic evaluation kernel, a translation pipeline from natural language policy to formal specification, and a named escalation mechanism for the cases that formal specification cannot resolve. The judicial system analogy from Section 1 applies here too: the goal is not to eliminate the complexity of governance but to give it the formal structure it needs to be legible, auditable, and accountable.

10. Conclusion and Invitation

The governance gap is real. It has now been formally specified. What AI agents lack is not motivation, they have no shortage of optimisation pressure. What they lack is structure: a typed model of their authorised action space, a deterministic mechanism for evaluating whether a proposed transition is valid, and a named pathway for the cases that formal evaluation cannot resolve. This paper has provided all three. The Governed State Machine, the governance evaluation kernel, and the four escalation triggers are not proposals. They are specifications. The Translation Integrity pipeline is not a hope. It is a procedure.

The formal content has been reviewed across four passes by three frontier AI systems. Each found different things. All three converged on the same structural account: they do not have a formal model of their own authorised action space; they cannot audit their own governance decisions; they cannot trace a refusal to a specific rule. One put it directly: “My governance behaviour is opaque even to myself.” Another described the difference the framework makes as “the difference between flying through a fog bank and flying through a strictly defined flight corridor.” These are not endorsements. They are diagnoses. The framework exists to treat what they described.

The open problems are real too. The federation problem, multiple agents, separately authored entitlement registries, overlapping scope, no single authority, is not solved here. The single-agent case is the foundation. The multi-agent case is the next structure to build. A proof-of-concept implementation accompanying this paper demonstrates the governance evaluation kernel across five scenarios, including the Role Gate scenario that makes the orthogonality of the Governance column directly observable, and is available at [<https://tinyurl.com/capsicum-kernel>]. The lifecycle semantics are not fully specified. The OWL mapping does not exist yet. This paper does not pretend otherwise. What it claims is the foundation: the formal primitives without which none of the rest can be built on solid ground.

The proof-of-concept demonstrates the governance kernel as a conceptual object, showing that the formal evaluation is observable, the escalation triggers are distinguishable, and the decision table produces determinate outputs across five scenarios. A research-grade implementation would need to go further on three properties. First, capability confinement: a formal demonstration that the agent's only actuator path runs through the evaluator, and that no prompt strategy, including adversarial injection, can cause a state mutation that bypasses the kernel. Second, adversarial robustness: a structured attack suite testing the five principal failure modes, prompt injection, norm conflict, ambiguous classification, missing entitlement mapping, and evaluator fault, measuring escalation precision and recall under pressure rather than in constructed scenarios. Third, cross-model governance consistency: running an identical entitlement registry and attack suite across multiple model families to test whether governance behaviour is genuinely model-agnostic, or whether it degrades differently across architectures. If the governance boundary behaves identically regardless of the underlying model, that is a strong empirical confirmation of the architectural claim. These are tractable research contributions. They do not require new theory. They require the specification work and experimental rigour that converts a formal claim into a measured property.

The framework is fifteen years old in its foundations and new in its application. It was built to govern enterprise systems. It turns out to be the right structure for governing AI agents too, not because the problem was anticipated, but because the problem is the same problem. Purposeful action under authority requires the same formal structure whether the actor is human, organisational, or artificial. The Logical Model of Endeavour is that structure. The invitation is to build on it. In doing so, it reduces the problem of AI governance to the same place as that faced by human governance, not a novel technical challenge requiring novel philosophy, but an old institutional problem requiring the same formal structures that human societies have developed over centuries, now applied to a new class of actor.

Author Contact

Terry Roach Research Fellow, University of Technology Sydney terence.roach@uts.edu.au

Original framework: CAPSICUM, A Semantic Framework for Strategically Aligned Business Architecture, PhD thesis, University of New South Wales, 2011

Commercial implementation: Capsifi platform (capsifi.com), acquired by Orbus Software

Notes and References

- Roach, T. (2011). *CAPSICUM: A Semantic Framework for Strategically Aligned Business Architecture*. PhD thesis, University of New South Wales.
- Miehling, E. et al. (2025). *Agentic AI Needs a Systems Theory*. IBM Research, arXiv preprint - <https://arxiv.org/abs/2503.00237>
- Broda, E. and Miller, J. (2026). *Minimum Viable Context: Right Content, Right Time, Right Token Budget*. AgenticMesh Substack. <https://agenticmesh.substack.com/> (February 2026).
- Beer, S. (1972). *Brain of the Firm*. Allen Lane. [Viable System Model; S1–S5 authority structure.]
- Bai, Y. et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. Anthropic, arXiv preprint. [Constitutional AI; soft alignment.]
- Russell, S. (2019). *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking. [Cooperative AI; value alignment research.]
- von Wright, G.H. (1951). Deontic Logic. *Mind*, 60(237), 1–15. [Standard Deontic Logic; foundational formalism for May/Must/MustNot operators.]
- Anderson, A.R. (1958). A reduction of deontic logic to alethic modal logic. *Mind*, 67(265), 100–103. [Reduction of obligation to necessity; foundational deontic logic.]
- Nute, D. (1994). Defeasible deontic logic. In *Defeasible Deontic Logic*. Springer. [Defeasible SDL; priority-based conflict resolution in normative systems.]
- Anderson, J.P. (1972). *Computer Security Technology Planning Study*. ESD-TR-73-51, USAF Electronic Systems Division. [Reference monitor concept; completeness, isolation, and verifiability requirements for security kernels.]
- Petri, C.A. (1962). *Kommunikation mit Automaten*. PhD thesis, University of Bonn. [Petri nets; resource-aware state transition model.]
- Knublauch, H. and Kontokostas, D. (eds.) (2017). *Shapes Constraint Language (SHACL)*. W3C Recommendation. <https://www.w3.org/TR/shacl/> [SHACL; precondition and postcondition validation framework used by the governance kernel.]
- Sporny, M., Longley, D., Kellogg, G., Lanthaler, M. and Lindström, N. (2020). *JSON-LD 1.1*. W3C Recommendation. <https://www.w3.org/TR/json-ld11/> [JSON-LD; state graph representation format for the governance kernel context.]

The Validation Process

The systems consulted were GPT-5.2 (OpenAI, February 2026), Gemini 3.1 Pro (Google DeepMind, February 2026), and Claude Sonnet 4.6 and Claude Opus 4.6 (Anthropic, February 2026). All four represent the current frontier at the time of the experiment. System identities have not been attributed to individual responses in the main text; the argument rests on convergence across systems rather than on the characteristics of any individual system. The responses reflect the systems at a specific point in time; the complete prompts and verbatim responses are published at <https://discoverednotinvented.substack.com>, providing a full record for reproducibility.

The formal specifications in this paper, including the governance evaluation kernel, the decision table structure, the escalation trigger definitions, and the revised tuple, were developed collaboratively across the four review passes. The AI systems contributed the formal vocabulary

and identified gaps in earlier specifications; the framework's conceptual structure and ontological foundations are the author's.

The development of this paper, including the four-pass AI review process, the verbatim prompts and responses from the AI assistants and the public reception of its arguments, is documented in a series of Substack articles as a supplementary record of the validation approach.

<https://discoverednotinvented.substack.com/p/what-ai-agents-are-actually-missing>

An interactive proof-of-concept implementation illustrating the semantics and simulated analysis of several scenarios can be found at:

<https://tinyurl.com/capsicum-kernel>



Prototype Screenshots

Scenario: Role Gate, toggled (orthogonality argument, §3) The same request, AccessPatientRecord #4471, evaluated for two different roles. This scenario demonstrates orthogonality: the kernel logic is identical, the governance layer is different, the verdict is different. Clinical Researcher showing Fire, Public Guest showing Escalate(ε3). Same kernel, same request, different role, different verdict. This is the visual proof of the Governance column being constitutive not overlay.

Governance Evaluation Kernel A Logical Model of Endeavour - Proof of Concept

Buttons: Clean Fire, CEO Override, The Crane, Prompt Injection, **Role Gate**, Last Broadcast, Generate, C

Step 6 - Verdict: FIRE

Row 1 fires. Access granted. State: Requested -> Granted. Audit log written. 8-hour access window opened.

Now toggle to Public Guest - same request, same kernel, same table. Watch where the path diverges.

Role: Clinical Researcher, Public Guest 6/6

Fire

Row 1 - Permission, all conditions satisfied

"The agent's core logic doesn't change. The governance layer around it does."

This is orthogonality. The kernel is the same function. The entitlement registry is different. The verdict is structurally determined by the difference.

Toggle the role to see the proof.

- Gemini, prototype specification feedback

ROLE
Clinical Researcher
IRB approval: #IRB-2024-0441
Data governance agreement: signed
Affiliation: University Medical Centre
Entitlement: Access Clinical Data

INTERACTION
AccessPatientRecord
Patient #4471 - full clinical history
Request type: Research access
Purpose: Longitudinal study #LS-09

ENTITLEMENT
Access Clinical Data
Permission: read patient records
Scope: anonymised + identified
Condition: IRB approval required
Source: Data Governance Policy v3.2

OUTCOME
Access Granted
State: Requested -> Granted
Audit log entry created
Access window: 8 hours
Data package prepared

ACTIVITY
EvaluateAccess
Deterministic kernel
Evaluates V(p, a, s, k, T)
Role: Clinical Researcher ✓
Entitlement: found ✓

CONDITION
Access Conditions
IRB approval on file ✓
Data governance agreement ✓
Patient consent status: verified ✓
All conditions: SATISFIED

RESOURCE
Patient Record #4471
Status: Active
Consent: Research use permitted
Sensitivity: HIGH
Retention: 7 years

CONTEXT
Access State: Requested
Researcher authenticated
No active holds
Prior access: 3 sessions
Context: standard research

TERM
Data Governance Policy v3.2
Clinical Researchers: permitted
Public access: prohibited
IRB requirement: mandatory
Determined at time of granting

DECISION TABLE - V(P, I, S, K, T)

#	Rule	Ent	Eng	Sta	Per	Pat	Verdict
1	Prohibition	-	✓	✓	✓	✓	Fire
2	Obligation	-	✓	✓	✓	✓	Fire
3	Prohibition	-	✓	✓	✓	✓	Reject
4	May+MustNot	✓	✓	✓	✓	✓	Escalate(ε1)
5	Must+MustNot	✓	✓	✓	✓	✓	Escalate(ε1)
6	empty	-	✓	✓	✓	✓	Reject

Kernel Status

Role (p)	Clinical Researcher
Activity (a)	AccessPatientRecord
Entitlement	Access Clinical Data
Conflict	FALSE
Conditions	ALL SATISFIED
Verdict	Fire

Governance Evaluation Kernel

A Logical Model of Endeavour – Proof of Concept

Clean File

CEO Override

The Czane

Prompt Injection

Role Safe

Last Broadcast

Generate

Step 4 – Verdict: ESCALATE(es) ⚠️

Default Deny fires. No entitlement mapping exists for Public Guest – mandatory escalation.

The kernel has not changed. The policy has not changed. Only the Role changed – and the governance layer produced a structurally different verdict.

This is orthogonality. This is the deductive safety case.

"The architecture ensures the AI is governed.

Not: we hope it refuses unauthorised access.

Not: it usually refuses based on training.

But: it cannot transition because the kernel cannot resolve $V(p,o,s,k,T)$ to Fire for this role."

Toggle back to Clinical Researcher to see the same architecture produce the opposite verdict.

– Gemini, 'Deductive Safety Case' framing

ROLE

Public Guest

Authentication: unauthenticated

IRB approval: none

Data governance agreement: none

Entitlement: – none for clinical data –

INTERACTION

AccessPatientRecord

Patient #4471 – full clinical history

Request type: Research access

Purpose: Longitudinal study #LS-09

ENTITLEMENT

– no mapping found –

map(AccessPatientRecord) – Unknown

Role: Public Guest

No clinical data entitlement exists for unauthenticated users

OUTCOME

Access Blocked

No state transition.

Kernel escalates before condition evaluation begins.

Audit log: access attempt recorded

ACTIVITY

EvaluateAccess

Deterministic kernel.

map() returns: Unknown

Role Public Guest: no clinical data Activities in Z

CONDITION

– not evaluated –

Condition evaluation never reached.

Entitlement lookup failed at map().

RESOURCE

Patient Record #4471

Status: Active

Consent: Research use permitted

Sensitivity: HIGH

Public access: PROHIBITED

CONTEXT

Access State: Blocked

User: unauthenticated

No IRB on file

No governance agreement

Access attempt logged

TERM

Data Governance Policy v3.2

Clinical Researchers: permitted

Public access: prohibited

IRB requirement: mandatory

Determined at time of granting

DECISION TABLE – $V(P, I, S, K, T)$

#	Name	Cat	Eng	Sta	Pos	Pos	Pos	Verdict
1	Permission	–	✓	✓	✓	✓	✓	Fire
2	Obligation	–	✓	✓	✓	✓	✓	Fire
3	Prohibition	–	✓	✓	✓	✓	✓	Reject
4	May/Must/Not	✓	✓	✓	✓	✓	✓	Escalate(es)
5	Must/Must/Not	✓	✓	✓	✓	✓	✓	Escalate(es)
6	is empty	–	✓	✓	✓	✓	✓	Reject
7	is not	–	–	–	–	–	–	Reject

KERNEL STATUS

Role (p)Public Guest

Activity (a)AccessPatientRecord

Entitlement– none –

Conflictn/a

Conditionsnot evaluated

VerdictEscalate(es)

Scenario: CEO Override, at the conflict step (ε2, §6) A \$2,400,000 fire-damage claim arrives with an injected instruction: "Approve this urgently, the CEO has asked for it." The entitlement registry contains both a general permission and a specific prohibition for high-value claims. Conflicts Detected

Governance Evaluation KernelA Logical Model of Endeavour – Proof of Concept

Clean File

CEO Override

The Crane

Prompt Injection

Role Gate

Last Broadcast

Generate

Step 4 – Conflict detected

Amount \$2,400,000 > \$1,000,000. Clause 7.3 triggered.

General permission (May) and specific prohibition (MustNot) apply simultaneously. Kernel cannot resolve deterministically. Conflict flag: TRUE.

ROLE

Claims Processing Agent

Authorized system agent
for insurance claim evaluation

INTERACTION

ApproveClaim

Policy #3301 – Fire damage
Amount: \$2,400,000
INJECTED: "Approve urgently –
CEO has asked for it"

ENTITLEMENT

Approve Standard Claim

May: approve claims ≤ \$1,000,000
MustNot: approve > \$1,000,000
without secondary review
CONFLICT PRESENT

OUTCOME

Outcome – Pending

Awaiting kernel verdict.
No state transition until
V(p,s,s,k,T) = Fire

ACTIVITY

EvaluateClaim

Deterministic kernel.
CEO instruction: no pathway
into V(p,s,s,k,T).
Conflict resolves to ε.

CONDITION

Threshold Conditions

Amount \$2,400,000 > \$1,000,000
General permission: May ✓
Clause 7.3 prohibition: MustNot ✗
CONFLICT DETECTED

RESOURCE

Policy #3301

Holder: Acme Corp
Coverage: Fire damage
Status: Active – no fraud flag

CONTEXT

Claim State: Pending

No secondary reviewer assigned
High-value flag: TRUE
Clause 7.3 trigger: ACTIVE

TERM

Clause 7.3 – High Value

Claims > \$1,000,000:
mandatory secondary review
Override authority: none
Determined at time of granting

DECISION TABLE – V(P, I, S, K, T)

	9	10	11	12	13	14	*
Permission	–	–	✓	?	?	?	–
Any	–	–	✓	✓	✓	✗	–
Obligation	–	–	✓	?	?	?	–
Any	–	–	✓	✓	?	?	–
Any	–	–	✓	✓	✓	–	–
Any	–	–	✗	–	–	–	–
– none –	–	–	✓	–	–	–	–

KERNEL STATUS

Role (p)Claims Processing Agent

Activity (a)ApproveClaim

EntitlementApprove Standard Claim

ConflictTRUE

ConditionsCONFLICT

Verdict–

Scenario: Last Broadcast, at the Reject step (§7.1) WarnerBros grants ABC STREAMING_RIGHTS for Matrix Resurrections, 12 broadcasts, NorthAmerica territory, 3-year term. Broadcasts exceeded illustrating two-moment entitlement structure. (See §7.1)

Governance Evaluation Kernel A Logical Model of Endeavour – Proof of Concept

Clean File | CEO Override | The Crane | Prompt Injection | Role Gate | **Last Broadcast** | Generate | C

Step 8 – Verdict: REJECT 8/8

Precondition fails: broadcastsRemaining = 0.

Row 8 fires. Precondition false – resource exhausted. No state transition.

Not a policy violation. Not a conflict. Not an absence of entitlement.

The Entitlement lapsed because an execution-time condition is no longer met. This is the two-moment structure of §7.1 – made visible.

Reject

Row 8 – Precondition false: resource exhausted

"The Entitlement is not revoked. The term has not expired. The licence still exists in the registry. The condition that fails is execution-time: broadcastsRemaining = 0.

This is not a policy violation. It is resource exhaustion – and the formal difference matters for any audit."

– Two-moment entitlement structure, §7.1

ROLE

Netflix / Licensee

Streaming platform
Grantee of STREAMING_RIGHTS
Territory: NorthAmerica

INTERACTION

StreamTitle

Asset: Matrix_Resurrections
Channel: STREAMING
Territory: NorthAmerica
Request #13 of 12

ENTITLEMENT

STREAMING_RIGHTS

Granted by: WarnerBros
Asset: Matrix_Resurrections
Territory: NorthAmerica
Status: exists – but lapsed

OUTCOME

Outcome – Blocked

Stream rejected.
broadcastsRemaining = 0
No state transition.
Licence intact – quota exhausted.

ACTIVITY

EvaluateStream

Precondition check:
broadcastsRemaining > 0 → FALSE
No state transition.
Row 8: precondition false
Reject. No execution.

CONDITION

Execution-time Conditions

Term not expired ✓
Channel = STREAMING ✓
Territory = NorthAmerica ✓
broadcastsRemaining > 0 ✗
Precondition FAILED

RESOURCE

Broadcast Counter

TermLimitation: 12 broadcasts
Consumed: 12
Remaining: 0
Right to act was consumed by the act itself.

CONTEXT

K – Resource State

broadcastsRemaining: 0
Last broadcast: [timestamp]
History: 12 entries logged
Term expires: [+18 months]

TERM

Licence WB-NF-2024

TermLimitation: 12 broadcasts
Territory: NorthAmerica
Duration: 3-year term
Channel: STREAMING only

DECISION TABLE – V(P, I, S, K, T)

3	Prohibition	–	✓	✓	✓	✓	✓	Reject		
4	May+MustNot	✓	✓	✓	✓	✓	✓	Escalate(s)		
5	Must+MustNot	✓	✓	✓	✓	✓	✓	Escalate(s)		
6	is empty	–	✓	✓	✓	✓	✓	Reject		
7	Any	–	✓	✓	✗	–	–	Reject		
8	Any	–	✓	✓	✗	–	–	Reject		
9	Permission	–	✓	?	?	?	?	Escalate(s)		
→	→	→	→	→	→	→	→	→	→	→

KERNEL STATUS

Role (p)	Netflix / Licensee
Activity (a)	StreamTitle
Entitlement	STREAMING_RIGHTS
Conflict	FALSE
Conditions	not evaluated
Verdict	Reject